Explainable Breast Cancer Detection using Ensemble Learning

Individual Contribution Report

EPICS Phase – II

Submitted by

Medhansh Singh Kushwaha
Register Number: 23BCE11385

in partial fulfillment of the requirements for the degree of

Bachelor of Technology
Computer Science and Engineering

VIT Bhopal University
Kothrikalan, Sehore
Madhya Pradesh

March 2026

Abstract

Breast cancer is still one of the biggest health challenges for women around the world, and detecting it early can genuinely save lives. Our team built a machine learning system that tries to make cancer detection not just accurate, but also transparent – because in healthcare, a black-box prediction isn't very helpful.

We experimented with multiple classifiers – Logistic Regression, Decision Trees, SVM, and Random Forest – and then combined them using soft voting to get predictions that are more stable and trustworthy. My job was specifically focused on this ensemble part.

One thing we really cared about was making the system explainable. We didn't want it to just spit out "malignant" or "benign" – we wanted confidence scores and insight into which features drove the decision. Our best models (Logistic Regression and SVM) both reached 98.25% accuracy with ROC-AUC above 0.99, which we were pretty excited about.

Keywords: Breast Cancer Detection, Ensemble Learning, Soft Voting, Model Evaluation, Performance Metrics, Classifier Combination

Contents

1 MY CONTRIBUTION **1**
- 1.1 Overview . . . 1
- 1.2 Ensemble Learning Implementation . . . 1
  - 1.2.1 Soft Voting Classifier . . . 1
  - 1.2.2 Why Soft Voting . . . 1
  - 1.2.3 Soft Voting Algorithm . . . 2
- 1.3 Performance Evaluation . . . 2
  - 1.3.1 Evaluation Metrics Implementation . . . 2
  - 1.3.2 ROC-AUC Calculation . . . 2
  - 1.3.3 Cross-Validation . . . 3
- 1.4 Results Analysis . . . 3
  - 1.4.1 Cross-Validation Results . . . 3
- 1.5 Ensemble Performance Analysis . . . 3
  - 1.5.1 What Worked Well . . . 4
  - 1.5.2 What Surprised Me . . . 4
- 1.6 Reliability and Robustness Assessment . . . 4
  - 1.6.1 Confidence Score Analysis . . . 4
  - 1.6.2 Error Analysis . . . 4

2 PROJECT OVERVIEW **5**
- 2.1 Problem Statement . . . 5
- 2.2 System Architecture . . . 5
- 2.3 Results Achieved . . . 5

3 TEAM CONTRIBUTIONS **6**
- 3.1 Team Members and Their Roles . . . 6
- 3.2 Collaboration . . . 6

4 LEARNING OUTCOMES **7**

5 CONCLUSION **8**

6 REFERENCES **9**

1. MY CONTRIBUTION

1.1 Overview

My main responsibility in this project was putting together the **ensemble learning system** – basically figuring out how to combine predictions from multiple models into one that's hopefully better than any of them alone. I also spent a lot of time working on the evaluation metrics to make sure we were measuring performance properly.

1.2 Ensemble Learning Implementation

1.2.1 Soft Voting Classifier

The core idea I implemented was soft voting, where instead of just counting "votes" from each model, we average their probability estimates. Mathematically, it looks like this:

$$\hat{y} = \arg\max_{c \in \{0,1\}} \sum_{j=1}^{M} w_j \cdot P_j(y = c|x) \tag{1}$$

Here, $M = 4$ (we have four models), and I used equal weights $w_j = \frac{1}{M}$ for simplicity.

Listing 1: Soft Voting Ensemble Implementation

```python
from sklearn.ensemble import VotingClassifier

def build_ensemble(models):
    """Build soft voting ensemble classifier."""
    estimators = list(models.items())
    ensemble = VotingClassifier(
        estimators=estimators,
        voting="soft"  # Use probability averaging
    )
    return ensemble
```

1.2.2 Why Soft Voting

I picked soft voting over hard voting for a few reasons:

- It uses probability estimates instead of just yes/no predictions, which is way more informative
- A model that's 95% confident should count more than one that's barely 51% confident
- We can actually get meaningful confidence scores out of it
- For medical stuff, knowing *how sure* the model is matters a lot

1.2.3 Soft Voting Algorithm

Here's the actual prediction flow I set up:

Algorithm 1 Soft Voting Prediction

Require: Input sample x, trained models $\{m_1, ..., m_M\}$, weights $\{w_1, ..., w_M\}$
Ensure: Predicted class $\hat{y}$, confidence $\hat{p}$

```
for j = 1 to M do
    p_j ← m_j.predict_proba(x)                    ▷ p_j ∈ ℝ^|C|
end for
p_avg ← Σ_{j=1}^{M} w_j · p_j                     ▷ Weighted average
ŷ ← arg max(p_avg)
p̂ ← max(p_avg)                                    ▷ Confidence score
return ŷ, p̂
```

1.3 Performance Evaluation

I also built out a pretty thorough evaluation system because I wanted to make sure we weren't fooling ourselves with just accuracy numbers.

1.3.1 Evaluation Metrics Implementation

These are the metrics I implemented and tracked:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \tag{4}$$

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5}$$

1.3.2 ROC-AUC Calculation

ROC curves were really useful for understanding performance across different thresholds:

Listing 2: ROC-AUC Calculation

```python
from sklearn.metrics import roc_curve, auc

def calculate_roc_auc(model, X_test, y_test):
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, thresholds = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)
    return fpr, tpr, roc_auc
```

1.3.3 Cross-Validation

To make sure our results weren't just from getting lucky with the train/test split, I ran 5-fold cross-validation:

$$CV_k = \frac{1}{k}\sum_{i=1}^{k} \text{Score}(m_i, \mathcal{D}_i^{test}) \tag{6}$$

Listing 3: Cross-Validation Implementation

```python
from sklearn.model_selection import cross_val_score

def evaluate_with_cv(model, X, y, cv=5):
    scores = cross_val_score(model, X, y, cv=cv)
    return scores.mean(), scores.std()
```

1.4 Results Analysis

After running everything, here's how the models stacked up:

Table 1: Performance Metrics Comparison

Model	**Accuracy**	**Precision**	**Recall**	**F1**	**ROC-AUC**
Logistic Regression	0.9825	0.9861	0.9861	0.9861	0.9954
SVM	0.9825	0.9861	0.9861	0.9861	0.9950
Ensemble	0.9737	0.9726	0.9861	0.9793	0.9954
Random Forest	0.9561	0.9589	0.9722	0.9655	0.9939
Decision Tree	0.9123	0.9559	0.9028	0.9286	0.9157

1.4.1 Cross-Validation Results

Table 2: 5-Fold Cross-Validation Results

Model	**Mean Accuracy**	**Std Dev**	**95% CI**
Logistic Regression	0.9802	0.0128	[0.9546, 1.000]
SVM	0.9714	0.0179	[0.9356, 1.000]
Ensemble	0.9692	0.0176	[0.9340, 1.000]
Random Forest	0.9538	0.0235	[0.9068, 1.000]
Decision Tree	0.9099	0.0189	[0.8721, 0.9477]

1.5 Ensemble Performance Analysis

Honestly, the results were a bit surprising – here's my take on them:

1.5.1 What Worked Well

- The ensemble kept recall at 98.61%, which is crucial – we really don't want to miss actual cancer cases
- It tied for best ROC-AUC (0.9954), showing excellent discrimination ability
- It's more stable than Decision Tree alone, which was kind of all over the place
- The confidence scores are actually useful, not just numbers

1.5.2 What Surprised Me

- The ensemble didn't beat Logistic Regression or SVM on accuracy – that was unexpected
- Turns out this problem is pretty linear once the data is preprocessed properly
- The ensemble does add safety though – if one model goes haywire, the others can compensate
- I think weighted voting (giving better models more say) could have improved results

1.6 Reliability and Robustness Assessment

I dug deeper into how reliable the system actually is:

1.6.1 Confidence Score Analysis

- When the model got things right, confidence was usually high (above 0.8)
- Wrong predictions tended to have confidence near 0.5 – basically the model was unsure
- This is actually good news – it means the confidence scores are meaningful

1.6.2 Error Analysis

I looked closely at the 3 samples we got wrong:

- 2 were false positives (benign tumors called malignant)
- 1 was a false negative (actual cancer we missed)
- All three had feature values right on the edge between classes – borderline cases

2. PROJECT OVERVIEW

2.1 Problem Statement

We set out to build something that could tell benign tumors from malignant ones, give confidence levels with predictions, and actually explain its reasoning. Just a "yes cancer" or "no cancer" output wasn't going to cut it for a medical application.

2.2 System Architecture

We split the project into layers:

1. **Data Layer**: Getting and cleaning the data
2. **Model Layer**: The individual classification algorithms
3. **Ensemble Layer**: Where my soft voting work came in
4. **Output Layer**: Metrics, visualizations, and explanations

2.3 Results Achieved

The final numbers were solid:

- LR and SVM both got 98.25% accuracy
- ROC-AUC above 0.99 for the best models
- 98.61% recall – we caught almost all the actual cancer cases
- Cross-validation showed results were consistent, not just luck

3. TEAM CONTRIBUTIONS

3.1 Team Members and Their Roles

- **Bhavya Sharma**: Got the dataset and explained what all the medical features mean
- **Suryanshi Ranawat**: Handled data cleaning and exploratory analysis
- **Ujjwal Mishra**: Implemented the actual ML algorithms
- **Medhansh Singh Kushwaha (Me)**: Built the ensemble system and evaluation framework
- **Soumyadeep Chakravarti**: Kept everything coordinated and did the final writeup

3.2 Collaboration

Everyone pulled their weight on this one. My ensemble code took the models Ujjwal built and combined them, so we had to work closely to make sure everything connected properly. The team would usually meet up to discuss results and figure out next steps.

4. LEARNING OUTCOMES

Here's what I took away from this project:

1. **Ensemble Learning**: I really get ensemble methods now – not just how to use them, but when they help and when they don't
2. **Model Evaluation**: Learned that accuracy alone is never enough. Precision, recall, F1, ROC-AUC – they all tell different parts of the story
3. **Cross-Validation**: K-fold CV went from something I'd read about to something I actually understand and use
4. **Statistics**: Got more comfortable with confidence intervals and understanding what standard deviation actually means for model reliability
5. **Healthcare Context**: In medical ML, missing a cancer case (false negative) is way worse than a false alarm – that really shapes how you design things
6. **scikit-learn**: Got way more comfortable with the library, especially the ensemble and metrics modules

5. CONCLUSION

My part of this project was all about the ensemble – combining multiple models to (hopefully) get better predictions than any single one. I built a soft voting classifier that averages probability estimates from four different models, and I set up a thorough evaluation system to measure how well everything worked.

The results were interesting. The ensemble kept recall really high (98.61%), which matters a lot when you're dealing with cancer detection – missing an actual case could be devastating. But honestly, it didn't beat Logistic Regression or SVM on pure accuracy. That taught me something: ensembles aren't magic, and sometimes a simple model on well-prepared data is hard to beat.

Still, I think the ensemble adds value. It's more robust – if one model has a bad day, the others can compensate. And the confidence scores it produces are actually useful for doctors to know how certain the system is.

This EPICS project taught me a lot about how to properly evaluate ML systems, not just train them. The healthcare angle made everything feel more real – these aren't just numbers, they represent actual diagnostic decisions that matter.

6. REFERENCES

References

[1] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[2] A. Ebrahim, M. Sedky, and A. Mesbah, "Enhancing breast cancer prediction through stacking ensemble and deep learning integration," *PeerJ Computer Science*, vol. 11, e2461, 2025.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: `https://scikit-learn.org`

[4] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[5] S. Chakravarti et al., "EPICS Project: Explainable Breast Cancer Detection," GitHub, 2026. [Online]. Available: `https://github.com/Soumyadeep-Chakravarti/EPICS_Project`